

iChat Pro 软件测试实践第二次实验报告

一、实验基本信息

项目	内容
课程名称	软件测试实践
实验名称	AI 辅助的软件工程全链路实践
项目名称	iChat Pro 跨平台安全即时通信系统
小组编号	7
实验日期	26.9.22
小组成员	叶伟智（121052024140）、许栢境（121052024142）、林弘（121052024144）、庞嘉铭（121052024146）
代码仓库	HeanX/iChat_Pro
需求基线	本报告第五章（V2.0）

二、实验背景

上一次实验课完成了初步需求讨论，本次实验将初步设想转化为可执行的软件工程方案。上学期 iChat Pro 已经实现账号、联系人、私聊、群聊、WebSocket、端到端加密、文件传输和设置中心等基础功能，因此本学期不以重复展示旧功能为主要成果。

本学期新增目标为：

1. 将现有系统升级为可安装的 Windows App 和 Android App；
2. 将系统部署到公网云服务器，实现不同设备和网络间的真实聊天；
3. 强化 E2EE、多设备和弱网可靠通信；
4. 建立黑盒、白盒、安全、性能和兼容性组成的产业级测试体系；
5. 使用 GitHub、CI/CD 和 AI 辅助工具贯通需求、开发、测试和交付过程。

三、实验目标

1. 提交经过评审的软件需求规格说明书和任务拆解材料。
2. 建立需求、Issue、分支、提交、PR、测试、缺陷和发布之间的追踪关系。
3. 明确代码规范、版本管理方式、测试工具和质量门禁。
4. 规范 AI 在需求分析、测试设计、代码实现和缺陷分析中的使用方式。

- 5. 明确小组成员角色、任务范围和交叉验收关系。
- 6. 为后续 Windows、Android、云部署和产业级测试工作提供统一执行基线。

四、本报告内容与交付范围

本报告作为本次实验的单文件提交材料，合并呈现以下内容：

- 1. 本学期需求范围、需求亮点和验收方向；
- 2. 从需求到交付的软件工程全链路；
- 3. AI 在需求、开发、测试和文档中的实践方式；
- 4. 代码规范、Git/GitHub 管理规则和质量门禁；
- 5. 当前测试工具、计划引入工具及测试策略；
- 6. 任务编号、小组角色和成员分工登记表。

本报告第五章作为本次实验的需求基线，并纳入评审所需的主要需求、验收口径、工程规范与分工信息，无须依赖其他附件阅读。

五、需求确定结果

5.1 核心新增需求

本学期需求分为五条主线：

- 已有聊天功能回归基线
- └ windows App 产品化
 - └ Android App 产品化
 - └ 公网云部署
 - └ E2EE 与可靠通信增强
 - └ 产业级测试与 CI/CD

5.2 已有功能基线

上学期已经完成的功能不作为本学期创新成果重复申报，但必须纳入回归测试，保证新架构和新客户端不会破坏原有能力。

功能域	已有功能	本学期处理方式
账号与身份	注册、登录、退出、个人资料和头像	作为多端登录与云端部署的回归基线
联系人	用户搜索、好友申请、联系人列表	补充权限、异常和多端一致性测试
私聊	文本消息、实时收发、历史记录	增强可靠性、状态同步和 E2EE 验证
群聊	建群、成员管理、群消息	增强成员变更后的加密和权限测试
文件传输	图片及文件发送、下载和转发	增加密文存储、大小边界和恶意文件测试
实时通信	Django Channels 与 WebSocket	迁移到公网 WSS、Redis 和正式 ASGI 服务
端到端加密	浏览器端密钥与消息加解密	强化多设备、密钥变化、信任和互操作测试

功能域	已有功能	本学期处理方式
设置与界面	主题、个人设置、聊天界面	适配 Windows、Android 和不同屏幕尺寸
AI Assistant	已有实验性入口或界面	作为可选扩展，不影响核心聊天验收

本学期所有新增功能必须与上述已有功能共同回归，不能以完成客户端封装为由降低原系统可用性。

5.3 Windows 与 Android 产品化

- Windows 端应形成可安装、启动、卸载的 `.exe` 应用；
- Android 端应形成可安装 APK，完成触屏、返回键、权限和生命周期适配；
- Windows、Android 和 Web 使用同一云端账号和业务数据；
- 三端应能够双向发送消息、同步状态和传输支持的加密文件；
- 客户端应提供连接状态、断线提示和自动重连；
- 私钥和登录凭据不得明文写入普通配置或日志。

5.4 公网云部署

- 使用公网域名、HTTPS 和 WSS；
- 使用 Nginx 与正式 Django ASGI 服务；
- 正式数据库由 SQLite 升级为 PostgreSQL；
- WebSocket Channel Layer 使用 Redis；
- 文件存储必须持久化；
- 建立环境隔离、秘密管理、日志、监控、备份、恢复和回滚流程。

5.5 E2EE 与可靠通信

- 客户端使用 ECDH、HKDF、AES-GCM 等机制完成加解密；
- 服务端不保存消息明文、文件明文、私钥或文件密钥明文；
- 私聊保存接收者密文和发送者独立密文副本；
- 群聊为每个有效成员保存独立密文；
- 使用客户端消息 ID 保证重试幂等；
- 支持发送中、已发送、已送达、已读和失败状态；
- 支持网络恢复后的自动重连和遗漏消息补取；
- 密钥变化不得自动继承旧密钥的可信状态；
- 多设备能力采用独立设备标识、设备密钥和设备撤销策略逐步实现。

5.6 产业级测试

- 黑盒测试：等价类、边界值、判定表、状态迁移、场景法和错误推测；
- 白盒测试：语句、分支、条件、基本路径、循环和异常路径；
- 专项测试：HTTP、WebSocket、E2EE、安全、性能、稳定性、弱网和兼容性；
- 客户端测试：Windows 安装/E2E、Android 权限/UI/生命周期；

- 运维测试：部署冒烟、健康检查、备份恢复和回滚；
- 质量目标：核心后端语句覆盖率目标 85%，分支覆盖率目标 75%，高风险逻辑目标 90%。

5.7 非功能需求与验收口径

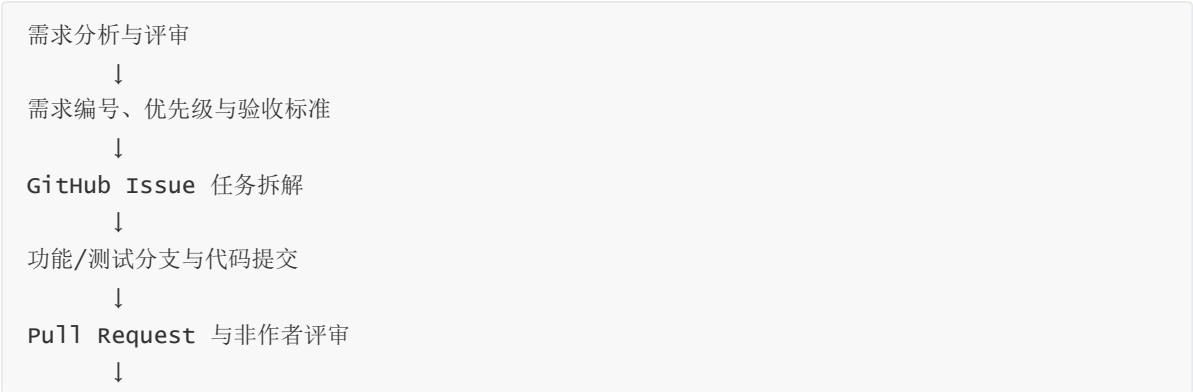
类别	主要要求	验收方式
可用性	三端提供一致的核心聊天流程，断线和失败状态清晰可见	跨端场景测试、用户操作检查
性能	常规负载下消息交互无明显卡顿，并形成并发测试基线	Locust/k6 报告、服务监控数据
安全性	HTTPS/WSS、最小权限、密钥不出端、无明文敏感日志	代码审查、ZAP/Bandit/依赖扫描、协议测试
可靠性	重连、重试、幂等、离线补取和消息状态可验证	弱网、断网、重复请求和恢复测试
兼容性	支持课程约定的 Windows、Android 和主流浏览器版本	真机/虚拟机兼容性矩阵
可维护性	任务、代码、测试和缺陷可追踪，配置与环境分离	Issue/PR/CI 记录审查
可恢复性	数据库和文件可备份恢复，发布失败可以回滚	恢复演练与回滚演练

5.8 需求亮点

1. **从 Web 原型走向真实产品**：形成 Windows 安装包、Android APK 和公网服务，而不是只在开发机演示。
2. **跨网络真实聊天**：不同设备通过 HTTPS/WSS 连接云端，验证消息和文件的真实收发。
3. **端到端加密**：服务端只中转和保存密文，重点验证密钥边界、多设备和成员变化场景。
4. **可靠通信**：针对重连、重试、幂等、离线补取和状态同步建立明确需求。
5. **产业级测试**：同时覆盖黑盒、白盒、接口、E2EE、安全、性能、兼容性和运维测试。
6. **AI 可审计实践**：记录 AI 输出、人工修改和真实验证，避免把 AI 生成内容直接当作成果。

六、软件工程全链路

6.1 总体流程



静态检查、自动化测试与覆盖率
↓
缺陷登记、修复与回归测试
↓
Windows/Android 构建与测试环境部署
↓
云端冒烟、发布审批与回滚准备
↓
实验报告、测试报告与演示证据

6.2 需求到 Issue

完整需求使用 `NEW-*`、`FR-*` 和 `NFR-*` 编号。任务按两套序列管理：

- 开发与工程任务：`P4 T01-P4 T54`；
- 测试任务：`P4 Test T01-P4 Test T24`。

当前已经在 GitHub 建立 78 个 Phase 4 Issues。每个 Issue 包含所属工作域、优先级、前置依赖、交付物和验收标准；本报告已完成四名成员的责任划分，后续将分工同步到 GitHub Assignee。

[查看 Phase 4 Issues](#)

6.3 Issue 到代码

每个开发或测试任务原则上建立独立分支：

```
feature/p4-t15-windows-package  
feature/p4-t24-android-build  
fix/p4-t32-message-idempotency  
test/p4-test-t13-websocket  
docs/p4-t54-final-materials
```

提交信息中引用 P4 Issue 编号，例如：

```
feat(android): build initial APK [P4 T24]  
fix(realtime): prevent duplicate resend [P4 T32]  
test(e2ee): add cross-platform vectors [P4 Test T06]
```

6.4 代码到评审

代码通过 Pull Request 合并。PR 至少说明：

- 关联 Issue；
- 修改目的和主要变更；
- 测试方法与测试结果；
- 界面或日志证据；
- 风险、兼容性和回滚方式。

原则上由一名非作者成员完成评审。E2EE、安全、权限、数据库迁移和部署配置等高风险改动应至少由开发和测试两个角色共同检查。

6.5 评审到测试

PR 合并前执行适用检查：

- 1. Django 系统检查；
- 2. migrations 一致性检查；
- 3. Python 后端测试；
- 4. JavaScript E2EE 测试；
- 5. 后续新增的格式、静态分析、覆盖率和依赖安全检查；
- 6. 对客户端或部署改动执行对应冒烟测试。

6.6 测试到缺陷闭环

缺陷使用 GitHub Issue 或缺陷记录表管理，内容必须包括：

- 关联需求和测试用例；
- 测试环境与前置条件；
- 复现步骤；
- 预期结果和实际结果；
- 严重程度和优先级；
- 截图、日志或录屏；
- 修复提交；
- 复测和回归结论。

6.7 发布与交付

发布流程分为测试环境和生产环境。测试环境完成自动部署和冒烟后，正式环境再进行人工审批。发布失败时按回滚文档恢复最近稳定版本，数据库变更必须具备向前兼容或恢复方案。

七、AI 实践

7.1 AI 使用场景

环节	AI 的作用	人工验证方式
需求分析	检查遗漏、冲突、歧义和不可测试需求	小组评审并对照代码与课程要求
任务拆解	将大型目标拆分为带依赖和验收标准的 Issues	检查工作量、优先级和关键路径
测试设计	提议等价类、边界、异常和安全场景	测试人员评审并在真实环境执行
测试代码	生成单元、接口或端到端测试初稿	运行、检查断言并进行代码评审
缺陷分析	汇总日志、推测根因和影响范围	人工复现、定位并通过回归确认

环节	AI 的作用	人工验证方式
文档维护	统一术语、编号、表格和追踪关系	与需求、Issues、提交和测试结果核对

7.2 AI 使用边界

1. AI 输出是建议，不直接作为需求通过或测试通过结论；
2. AI 生成的代码和测试必须经过人工评审并真实执行；
3. 不向外部 AI 服务提交生产密码、Token、私钥、用户隐私和 E2EE 聊天明文；
4. 不使用 AI 伪造测试截图、执行日志、覆盖率或缺陷结果；
5. 对 AI 输出中的错误、遗漏和人工修改形成记录；
6. 最终技术判断和交付责任由小组成员承担。

7.3 AI 实践证据

本阶段已经形成以下可追踪记录：

日期	关联任务	AI 辅助目标	输出	人工检查与验证	结论
2026-09-15	P4 T01-T03	区分已有功能和本学期新增需求	需求结构、亮点和验收口径草案	对照仓库现状与课程要求，修订为五条主线	修改后采用
2026-09-15	P4 T01-T54、P4 Test T01-T24	将需求拆成可开发、可测试、可验收任务	54 个工程任务、24 个测试任务	检查编号、依赖、范围及验收条件，并在 GitHub 建立 Issues	采用
2026-09-22	P4 T51-T54	整理软件工程全链路材料	报告结构、规范、工具和分工草案	区分“已使用”和“计划引入”，删除无法证实的完成性描述	修改后采用

后续每次 AI 辅助活动至少记录日期、执行人、Issue/PR、输入目标、输出摘要、人工修改、验证方法、证据链接和最终结论。AI 推测不得记录为已经复现、已经通过或已经完成。

八、代码管理工作介绍

8.1 源代码管理

- Git：本地版本、分支和提交管理；
- GitHub：远程仓库、Issues、Pull Requests 和 Actions；
- `main`：保持可运行、可测试的主分支；
- `.gitignore`：排除虚拟环境、依赖、数据库、媒体、缓存和本地配置。

8.2 当前自动化基础

仓库当前的 GitHub Actions 工作流在推送到 `main` 和 Pull Request 时执行：

```
安装 Python 3.13
→ 安装 requirements.txt
→ python manage.py check
→ python manage.py makemigrations --check --dry-run
→ python manage.py test
```

该流程是本学期 CI/CD 的起点，后续逐步加入格式检查、覆盖率、E2EE 测试、安全扫描、客户端构建和部署冒烟。

8.3 编码规范

Python 与 Django

- 遵循 PEP 8；变量和函数使用 `snake_case`，类使用 `PascalCase`，常量使用 `UPPER_SNAKE_CASE`；
- 导入按标准库、第三方库、项目模块分组，禁止无必要的通配符导入；
- View、Consumer、Service 和 Model 职责清晰，复杂业务逻辑不堆积在视图层；
- 数据模型变更必须提交 migration，并执行迁移一致性检查；
- 对象级操作必须检查当前用户权限，不能只依赖前端隐藏按钮；
- 涉及多表写入、消息状态和关键安全操作时按需使用事务；
- ORM 查询避免明显的 N+1 问题，异常响应不得泄露敏感信息。

JavaScript 与客户端

- 默认使用 `const`，确需重新赋值时使用 `let`，不使用 `var`；
- 变量和函数使用 `camelCase`，类或组件使用 `PascalCase`；
- 网络、WebSocket、加解密和本地存储操作必须处理失败、超时和重试；
- 不可信文本进入 DOM 前必须安全转义；
- Windows、Android 和 Web 保持协议字段一致，协议变化同步更新兼容性测试；
- 日志不得输出消息明文、密钥材料、密码或完整身份凭据。

通用与安全规范

- 文件统一使用 UTF-8，命名表达实际业务含义；
- 密码、Token、私钥和生产配置通过环境变量或秘密管理服务注入；
- 上传文件检查扩展名、MIME、大小和存储路径；
- 所有敏感接口验证身份、权限、请求方法、输入长度和类型；
- 调试输出、数据库、媒体文件、缓存和个人环境配置不得进入正式提交。

8.4 分支与提交规范

类型	命名示例	用途
功能分支	feature/p4-t14-windows-shell	新功能开发
测试分支	test/p4-test-t08-offline-replay	测试代码和测试环境
修复分支	fix/p4-t32-message-idempotency	缺陷修复
文档分支	docs/p4-t54-final-materials	文档与过程资产

提交信息采用“类型 + 简述 + Issue 编号”，例如：

```
feat: add windows reconnect indicator (P4 T18)
fix: reject unauthorized group message request (P4 T34)
test: cover offline message replay (P4 Test T08)
docs: update cloud deployment criteria (P4 T05)
```

常用类型为 feat、fix、test、docs、refactor、chore 和 ci。每次提交保持小而完整，不混入无关格式化和个人文件。

8.5 Pull Request 与完成定义

PR 必须写明关联 Issue、改动边界、测试方法、测试结果、风险和回滚方式；涉及界面时提供截图，涉及协议、数据库或配置时说明兼容性与部署步骤。

任务只有满足以下条件才可标记完成：

- 1. 验收标准全部满足；
- 2. 代码、配置、迁移和文档已经提交；
- 3. 相关自动化测试通过，人工测试有证据；
- 4. 非作者成员完成评审，阻断意见已经处理；
- 5. 不存在未说明的严重回归和高危安全问题；
- 6. 可以从需求追溯到 Issue、PR、测试和结果；
- 7. 部署类改动提供升级、兼容和回滚说明。

九、测试工具

9.1 当前已经使用

工具	当前用途
Django TestCase	模型、表单、视图、API 和权限测试
TransactionTestCase	实时通信、事务和 WebSocket 相关测试
Node.js	私聊和群聊 E2EE JavaScript 测试
Django system check	检查项目配置与注册组件
Django migrations check	检查模型变更是否遗漏迁移

工具	当前用途
GitHub Actions	自动安装依赖并执行 Django 检查和测试

9.2 本学期计划引入

工具	计划用途
coverage.py	Python 语句与分支覆盖率
Ruff	Python 格式和静态质量检查
Playwright	浏览器与 Electron 端到端测试
Appium	Android UI、权限和生命周期测试
Locust 或 k6	HTTP/WebSocket 性能测试
OWASP ZAP	Web 动态安全测试
Bandit	Python 静态安全分析
pip-audit / npm audit	第三方依赖漏洞检查
PostgreSQL / Redis	生产依赖集成测试

计划工具只有在完成安装、配置和真实执行后，才能写入最终测试成果。

9.3 测试方法与层级

测试类型	重点方法	主要对象
黑盒测试	等价类、边界值、判定表、状态迁移、场景法	登录、联系人、私聊、群聊、文件、设置
白盒测试	语句、分支、条件、路径和异常流	权限判断、消息状态、密钥处理、重试逻辑
接口测试	正常、异常、鉴权、幂等、参数边界	HTTP API 与 WebSocket 事件
E2EE 测试	测试向量、篡改、错密钥、跨端互操作	私聊、群聊、文件和多设备密文
安全测试	越权、注入、XSS、CSRF、依赖漏洞和敏感信息	Web、服务端、客户端与部署配置
性能测试	并发、吞吐、延迟、长连接和稳定性	登录、消息发送、WebSocket、数据库
兼容性测试	系统版本、屏幕、权限、浏览器和网络环境	Windows、Android、Web
运维测试	冒烟、健康检查、备份恢复和回滚	云端测试与正式环境

9.4 测试证据与准入规则

每次测试记录提交 SHA、环境、用例范围、执行人、时间、通过/失败/阻塞数量、日志和证据链接。测试数据使用构造数据，不公开真实聊天内容和密钥。

PR 合并前，相关自动化检查必须通过；发布前不得存在阻断级缺陷，严重缺陷必须修复或形成书面风险处置。失败用例不得通过删除测试、弱化断言或篡改数据规避。

十、小组分工方案

小组共 4 人，按照模块关联性合并角色，并将 78 个 Issue 均衡分配为 20、19、20、19 项：

角色	Issue 范围	主要责任
叶伟智：组长/需求、Windows、AI与文档负责人	P4 T01-T03、T14-T21、T51-T54；P4 Test T01-T03、T15、T24	需求与计划协调、Windows 产品化、Windows E2E、AI 记录和材料一致性
许栢境：云端、后端与设备服务负责人	P4 T04-T13、T31-T32、T40-T41；P4 Test T13-T14、T18-T19、T23	云端架构、数据服务、协议幂等、设备服务、集成性能和部署冒烟
林弘：Android、E2EE 与安全负责人	P4 T22-T30、T38-T39、T42-T43；P4 Test T04-T08、T16-T17	Android 产品化、威胁模型、密钥安全、跨端加密验证和安全测试
庞嘉铭：可靠通信、测试与 CI/CD 负责人	P4 T33-T37、T44-T50；P4 Test T09-T12、T20-T22	可靠通信、质量门禁、黑白盒、覆盖率、兼容性、缺陷和测试总结

每个 Issue 设置一名主负责人，可以设置协作者，但必须明确最终交付责任。开发成果由测试角色交叉验收，安全和 E2EE 改动至少经过两人评审。

10.1 成员任务登记表

姓名	学号	主要角色	主负责 Issue	协作/验收 Issue	计划交付物
叶伟智	121052024140	组长/需求、Windows、AI与文档负责人	P4 T01-T03、T14-T21、T51-T54；P4 Test T01-T03、T15、T24 (20 项)	统筹并验收 P4 T44-T50、P4 Test T21-T23	需求基线、Windows 安装包、E2E、AI 记录和最终材料
许栢境	121052024142	云端、后端与设备服务负责人	P4 T04-T13、T31-T32、T40-T41；P4 Test T13-T14、T18-T19、T23 (19 项)	协作验收 P4 T33-T37、T44-T50	云端架构、数据服务、消息幂等、设备服务和部署验证

姓名	学号	主要角色	主负责 Issue	协作/验收 Issue	计划交付物
林弘	121052024144	Android、E2EE 与安全负责人	P4 T22-T30、T38-T39、T42-T43；P4 Test T04-T08、T16-T17 (20 项)	安全复核 P4 T04-T21、T31-T41	APK、移动端适配、威胁模型、密钥安全和加密兼容报告
庞嘉铭	121052024146	可靠通信、测试与 CI/CD 负责人	P4 T33-T37、T44-T50；P4 Test T09-T12、T20-T22 (19 项)	独立验收其他成员提交的核心功能	可靠通信、流水线、黑白盒、覆盖率、兼容性、缺陷和测试总结

10.2 协作与验收机制

- 每个 Issue 指定一名主负责人和至少一名验收人；
- 开发人员不单独验收自己的关键功能，测试负责人保持相对独立；
- 成员在 Issue 中记录领取时间、预计完成时间、依赖和阻塞；
- 至少在开始、提交 PR、测试完成和出现阻塞时更新 Issue；
- 安全、E2EE、生产部署和发布操作至少由两人复核；
- 分工发生变化时同步更新 Issue Assignee 和本表。

十一、实验产出

本次实验形成了以下可检查成果：

1. 本学期需求基线、验收口径与需求亮点；
2. 需求亮点、范围、风险和任务拆解；
3. 54 个开发/工程 Issues；
4. 24 个测试 Issues；
5. AI 辅助软件工程的使用规则和记录模板；
6. 代码规范、分支规范、提交规范和 PR 规范；
7. 当前测试工具与计划工具清单；
8. 基于角色和 Issue 范围的小组分工方案；
9. 从需求到发布的全链路流程和证据要求。

十二、后续工作

1. 将本报告中的成员分工同步到各 GitHub Issue 的 Assignee；
2. 执行 P4 Test T01，记录当前代码和自动化测试基线；
3. 从 P4 T04 开始形成云端部署设计；
4. 完成 Windows、Android 技术可行性原型；
5. 配置覆盖率与静态检查，扩展 GitHub Actions；

6. 按本报告第七章持续保存 AI 使用及人工验证证据。

十三、结论

本次实验将上一次课的初步需求转化为正式需求基线、GitHub Issues、代码管理规范、测试工具体系和小组分工框架。AI 被纳入需求审查、任务拆解、测试设计和缺陷分析，但所有输出都必须经过人工评审和真实执行验证。由此，iChat Pro 后续工作可以按照统一编号和质量门禁，从需求持续推进到开发、测试、部署和交付。